



**POLITECNICO
DI TORINO**
Dipartimento di
Elettronica e
Telecomunicazioni



DIGITAL SYSTEMS ELECTRONICS

Academic year 2024-2025

LABORATORY NR. 1

DUE DATE: March 17, 2025

DELIVERY DATE: March 13, 2025

GROUP 20

CONTRIBUTIONS:

- Erfan Taheri
- Raman Jahandari
- Bardia Vafadar
- Ali Mansouri

The members of the group listed above declare under their own responsibility that no part of this document has been copied from other documents and that the associated code is original and has been developed expressly for the assigned project.

1 Introduction

The purpose of this lab is to familiarize ourselves with the *VHDL language (ModelSim and Quartus Prime)* to learn how to digital design multiplexer and switch on an FPGA by using VHDL and testing it on the given *DE1-SoC*. At the beginning we had trouble creating a new project in Quartus and assigning the pins due to the lack not familiarity with these circumstance however with good teamwork we completed all of the parts in the Laboratory assignment.

We defined and tested every part as we described in the following sections, in the first stage we defined the system, controlling the red LEDs that provided by the DE1 boards via 10 toggle switches.

in the second and third stage we used two different *Multiplexer*:

- In the second stage, we used a 2-to-1 Multiplexer, which means a multiplexer with two inputs and one output.
- In the third stage, we implemented a three-bit wide 5-to-1 multiplexer.

We also included some lines of code to better describe the model implemented, as well as the waveforms result given from the simulation.

2 Controlling the LEDs

The first circuit has the purpose of showing the status of the switches through the LEDs in the DE1-SoC. The design VHDL file has two parts, entity, and architecture which are there to define the input output signals and the logics respectively.

```
ENTITY LED_switch_control IS
    PORT (
        SW  : IN STD_LOGIC_VECTOR(9 DOWNTO 0);  — Switches (inputs)
        LEDR: OUT STD_LOGIC_VECTOR(9 DOWNTO 0)   — LEDs (outputs)
    );
END LED_switch_control;
```

where the **SW** represents as 10-bit input vector representing 10 switches(SW_{9-0}) and **LEDR** as output where represent the 10 LEDs.

After that, we assigned the switch values to LEDs.

```
ARCHITECTURE Behavior OF LED_switch_control IS
BEGIN
    LEDR <= SW;
END Behavior;
```

Now, each switch is connected to the condensing LED. It means that flipping a switch turns its corresponding LED to ON or OFF.

Then after compiling with Quartus, the pins locations were assigned to each signal, using the manual file of DE1-SoC.

In the next file `LED_Switch_tb.vhd` we simulate the LED-switch circuit using ModelSim the code is represented in the following

```
BEGIN
  — Connect signals to the component
  DUT: LED_switch_control PORT MAP (SW => SW, LEDR => LEDR);

  — Test process
  PROCESS
  BEGIN
    — Test Case 1: All switches off
    SW <= "0000000000";
    WAIT FOR 10 ns;

    — Test Case 2: SW[0] ON
    SW <= "0000000001";
    WAIT FOR 10 ns;

    — Test Case 3: SW[9:0] Alternating pattern
    SW <= "1010101010";
    WAIT FOR 10 ns;

    — Test Case 4: All switches ON
    SW <= "1111111111";
    WAIT FOR 10 ns;

    — End simulation
    WAIT;
  END PROCESS;
END test;
```

where the SW start with `SW <= "0000000000"` as turning OFF all 10 LEDs, after 10 ns, it moves to the next step which is the first LED (`SW <= "0000000001"`) turns ON and after 10 ns all LEDs turn ON (by `SW <= "1111111111"`).

After writing the testbench, we added it to the original project and then started the functional simulation. we also illustrate the results using waveforms to check for the correct functionality of the design. You can see the results of the process in Figure 1.

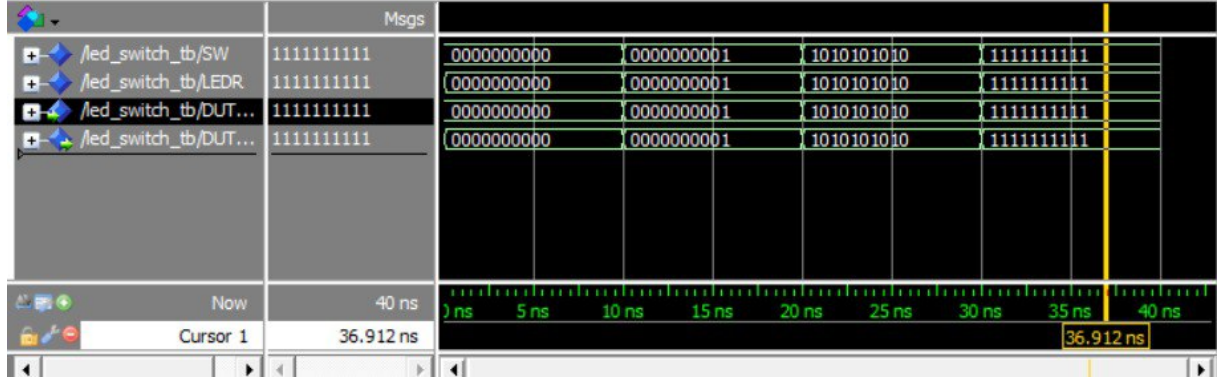
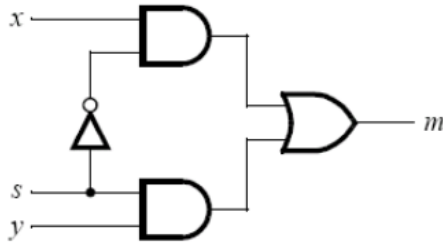


Figure 1: Test bench simulation results for LED control

3 2-to-1 Multiplexer

In the second part we implemented a 4-bit-2-to-1 multiplexer.

First we defined the entity **mux4tol**, introducing two 4-bit input vectors **X,Y** which represent the two input signals and a 4-bit output **M**. Additionally we define **S** a 1-bit input control signal that determines which input is forwarded to the Output. The following table in Figure 2 shows the elementary logic structure for better understanding:



1.Circuit

s	m
0	x
1	y

2.Truth table

Figure 2: On the left, Circuit design 2 to 1 multiplexer, using logic elements, On the right, the resulting truth table of the design

In the VHDL "**mux2to1.vhd**" the multiplexer logic is implemented. In the entity, we assigned **S** to SW_8 , first input **X** assigned to SW_{3-0} , second input **Y** assigned to SW_{7-4} and the output **M** as **LEDR** assigned to $LEDR_{3-0}$:

```

ENTITY mux2to1 IS
    PORT (
        S : IN  STD_LOGIC;           — 1-bit select input
        X : IN  STD_LOGIC_VECTOR(3 DOWNTO 0); — 4-bit input X ( $SW_{3-0}$ )
        Y : IN  STD_LOGIC_VECTOR(3 DOWNTO 0); — 4-bit input Y ( $SW_{7-4}$ )
        M : OUT STD_LOGIC_VECTOR(3 DOWNTO 0) — 4-bit output M ( $LEDR_{3-0}$ )
    );
END mux2to1;
```

In architecture, we define each bit of **M** output vector using the **Boolean logic** that ensures when **S=0**, the output matches to **X** and when **S=1**, the output matches to **Y**.

```

ARCHITECTURE Behavior OF mux4tol IS
BEGIN
    — Multiplexer logic using BOOLEAN
    M(0) <= (NOT S AND X(0)) OR (S AND Y(0));
    M(1) <= (NOT S AND X(1)) OR (S AND Y(1));
    M(2) <= (NOT S AND X(2)) OR (S AND Y(2));
    M(3) <= (NOT S AND X(3)) OR (S AND Y(3));
END Behavior;
```

The **Testbench** used to test the functionality of the circuit is defined as **mux2to1_4bit_tb.vhd**. In the ARCHITECTURE, first we set the initial states such as S define to "0" , X define to "0000" and Y "1111" . Then we added our multiplexer as a component

```

ARCHITECTURE test OF mux2to1_4bit_tb IS
SIGNAL S      : STD_LOGIC := '0';
SIGNAL X      : STD_LOGIC_VECTOR(3 DOWNTO 0) := "0000";
SIGNAL Y      : STD_LOGIC_VECTOR(3 DOWNTO 0) := "1111";
SIGNAL M      : STD_LOGIC_VECTOR(3 DOWNTO 0);

    — Component Declaration
    COMPONENT mux2to1
        PORT (
            S      : IN  STD_LOGIC;
            X      : IN  STD_LOGIC_VECTOR(3 DOWNTO 0);
            Y      : IN  STD_LOGIC_VECTOR(3 DOWNTO 0);
            M      : OUT STD_LOGIC_VECTOR(3 DOWNTO 0)
        );
    END COMPONENT;
```

Then implemented the process that sequentially toggles the control signal S, switching between X and Y as the inputs:

```

BEGIN
    — Instantiate the MUX
    DUT: mux4tol PORT MAP (S => S, X => X, Y => Y, M => M);

    — Test process
    PROCESS
    BEGIN
        — Case 1: S = 0
        S <= '0'; X <= "1010"; Y <= "0101"; WAIT FOR 10 ns;

        — Case 2: S = 1,
        S <= '1'; WAIT FOR 10 ns;
```

```

— Case 3: S = 0,
S <= '0'; X <= "1100"; WAIT FOR 10 ns;

— Case 4: S = 1,
S <= '1'; Y <= "0011"; WAIT FOR 10 ns;

— End simulation
WAIT;
END PROCESS;
END test;

```

The result of the waveform simulation can be seen in Figure 3.

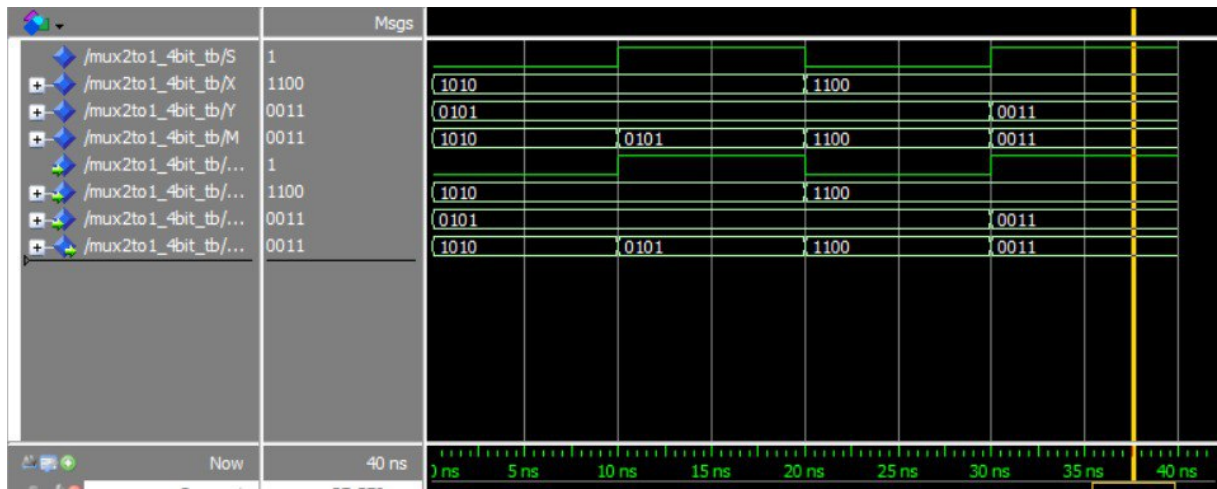
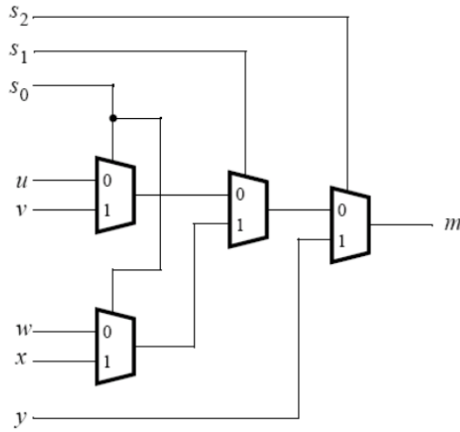


Figure 3: Simulation results for 2 to 1 multiplexer

4 5-to-1 Multiplexer

This part is implementing a 5-to-1 multiplexer (mux5to1.vhd) in VHDL, involving 5 three-bit input signals **u**, **v**, **w**, **x**, **y** and three one-bit control signals S_1, S_2, S_3 and a three-bit output **M**. The suggested circuit design and the corresponding truth table is shown in Figure 4.



1.Circuit Diagram

s_2	s_1	s_0	m
0	0	0	u
0	0	1	v
0	1	0	w
0	1	1	x
1	0	0	y
1	0	1	y
1	1	0	y
1	1	1	y

2.Truth Table

Figure 4: Circuit design 2 to 1 multiplexer, using logic elements

The first three signals are fixed to the values $\mathbf{u}=\mathbf{"101"}$, $\mathbf{v}=\mathbf{"010"}$, $\mathbf{w}=\mathbf{"111"}$ as was given from lab instruction. The $\mathbf{Y}$ signal is linked to the first three switches SW_{2-0} and the $\mathbf{X}$ signal is linked to the switches SW_{5-3} . Finally, the control signals are linked to the switches SW_{8-6} .

```

ENTITY mux5to1 IS
PORT (
    S  : IN  STD_LOGIC_VECTOR(2 DOWNTO 0);  — 3 bit select input (SW[8:6])
    X  : IN  STD_LOGIC_VECTOR(2 DOWNTO 0);  — 3 bit input X (SW[5:3])
    Y  : IN  STD_LOGIC_VECTOR(2 DOWNTO 0);  — 3 bit input Y (SW[2:0])
    M  : OUT STD_LOGIC_VECTOR(2 DOWNTO 0)    — 3 bit output M (LEDR[2:0])
);
END mux5to1;

```

And also we defined $\mathbf{U,V,W}$ in the following lines:

```

ARCHITECTURE Behavior OF mux5to1 IS
    — Declare constant values
    CONSTANT U : STD_LOGIC_VECTOR(2 DOWNTO 0) := "101";
    CONSTANT V : STD_LOGIC_VECTOR(2 DOWNTO 0) := "010";
    CONSTANT W : STD_LOGIC_VECTOR(2 DOWNTO 0) := "111";
    ....

```

In the architecture, the three inputs are defined as constants, which means that they cannot be changed throughout the code. In the process, the behavior of the 5-to-1 multiplexer is defined, using conditional statements (**IF**, **ELSIF**, and **ELSE**). This approach allows the multiplexer to select one of the five input signals $\mathbf{u}$, $\mathbf{v}$, $\mathbf{w}$, $\mathbf{x}$, $\mathbf{y}$ based on the value of the 3-bit control signal $\mathbf{s}$. The two golden rules of the process have been met, which are having the signals that are on the right side of the code in the sensitivity list of the process, and assigning a value to all of the outputs and not leaving any case behind, using the last line of **ELSE** code.

```

BEGIN
  PROCESS(S, X, Y)
  BEGIN

    IF S = "000" THEN
      M <= U; — Select constant "101"

                                ELSIF S = "001" THEN
      M <= V; — Select constant "010"

    ELSIF S = "010" THEN
      M <= W; — Select constant "111"

    ELSIF S = "011" THEN
      M <= X; — Select X (SW[5:3])

    ELSIF S = "100" THEN
      M <= Y; — Select Y (SW[2:0])

    ELSE
      M <= Y; — Other cases , which should be the Y

    END IF ;
  END PROCESS;

END Behavior;

```

The testbench file **mux5to1_tb.vhd** is used to verify the functionality of the 5-to-1 multiplexer. Adding mux5to1 as a component and applying test signals to the inputs **X**, **Y** and the control signal **S**. The process changes the value of S every 10 ns to simulate different control signal combinations. The resulting waveforms of the simulation with this test bench is shown in Figure 5.

```

ARCHITECTURE test OF mux5to1_tb IS
  SIGNAL S      : STD_LOGIC_VECTOR(2 DOWNTO 0) := "000";
  SIGNAL X      : STD_LOGIC_VECTOR(2 DOWNTO 0) := "000";
  SIGNAL Y      : STD_LOGIC_VECTOR(2 DOWNTO 0) := "111";
  SIGNAL M      : STD_LOGIC_VECTOR(2 DOWNTO 0);
  ...

BEGIN
  — Instantiate the MUX
  DUT: mux5to1 PORT MAP (S => S, X => X, Y => Y, M => M);

  — Test process
  PROCESS
  BEGIN
    — Test each select input

```



```

S <= "000"; WAIT FOR 10 ns; — Should select "101"
S <= "001"; WAIT FOR 10 ns; — Should select "010"
S <= "010"; WAIT FOR 10 ns; — Should select "111"
S <= "011"; X <= "100"; WAIT FOR 10 ns; — Should select X = "100"
S <= "100"; Y <= "011"; WAIT FOR 10 ns; — Should select Y = "011"

— End simulation
WAIT;
END PROCESS;
END test;

```

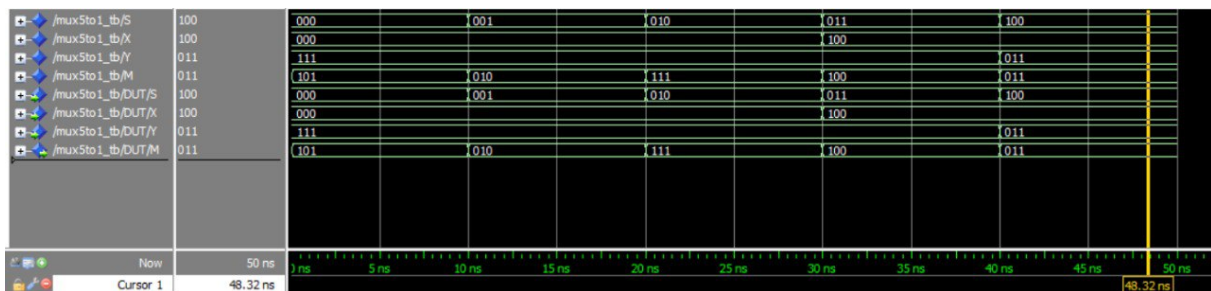


Figure 5: Simulation results for 5 to 1 multiplexer

5 Conclusion

Overall, this laboratory experience helped us learn the basics of VHDL's syntax, designing, and testing. The challenges we faced at the beginning were how to use Quartus and Modelsim. However, we handle it with teamwork and the instructions of the lab assignment. When it came to loading the code on the FPGA, we encountered some difficulties, such as how to find the device on the programmer section of the Quartus. These issues were solved in the lab session, by asking the lab assistants.

This laboratory has been a valuable learning experience that has helped us understand the process of digital circuit design and testing, and we look forward to implementing these skills in more complex projects.